NPTEL

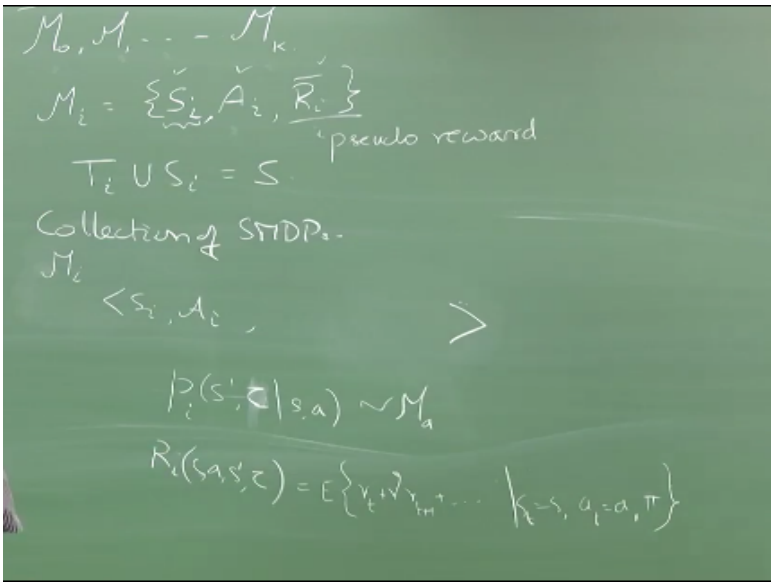
NPTEL ONLINE COURSE

REINFORCEMENT LEARNING

MAXQ Value Function Decomposition

Prof. Balaraman Ravindran
Department of Computer Science and Engineering
Indian Institute of Technology Madras

(Refer Slide Time: 00:19)



So you can think of there being a Ti, okay so I have specified the states where it will be active in so whatever is there in the state space which is not there in Si it is basically Ti and Ti are the states it will terminate in, okay in fact they have made a specific dichotomy like this so that essentially means that if you are in a terminal state and call an action call a subtask it will not move basically it will just return right.

So because it is not in the only in the active states do I specify actions if I am in one of the terminal states it won't I would not specify actions right on the contrary in options right the same state can be in I and the beta right in options the same state can be in I and in beta so I can start an action option in a state there might be also some probability of ending the option in the state right but if I start the option I will take at least one action.

So that is the thing so if I start from here I will not end here I can take an action go somewhere else right and then but if I come to the state while executing the option I might also terminate here does that make sense right, I have some state if I start an option here then I will certainly move out of the state okay if I start an option I will certainly move of that state but while executing the option if I land here I will stop okay.

So there is a little flexibility in the option definition that allows you to do that, okay any questions so now let us take a look at value functions how how do value functions work here, so you recall what I said right these are essentially collection of SMDPs it is a collection of SMDP so let me take one of those right let me take Mi right so we have Si Ai $\widehat{Ri}$ right so for the Mi SMDP, the SMDP corresponding to Mi so I have the states right so I have the actions right do I have the rewards $\widehat{Ri}$ + Ri, R R right the original reward both together give me the reward function for the SMDP.

So $\widehat{Ri}$ $\overline{Ri}$ + R gives me the rewards for the SMDP so so the SMDP corresponding to Mi so I have Si, Ai right what about the p, p is a little tricky so now p I have to define this joint distribution right, so if I only take the ahh okay I apologize yeah so we will come to that also what is that where are you getting the $\overline{Ri}$ from that is the task specification right when I specify the Mi those are the things I have to specify I have to specify Si, Ai, and $\overline{Ri}$.

Ok just like options you have to specify even option policy in HAMs you have to specify the structure of the machine so in Max Q you not only specify this graph right I mean this graph is

implicitly specified there right this graph is drawn from this Ai right, so basically have to specify Si Ai $\overline{Ri}$ for every task okay, + 5 ahh ok wrong destination that has to be part of $\overline{Ri}$ right.

Maybe when you drop off the passenger you might get a reward the task is for you to drop off the passenger at the goal right so for dropping off the passenger at the goal you might get a core MDP reward or for doing a wrong pick up you know when you try to pick up a passenger when there is no passenger or you are trying to pick up a passenger when there is already a passenger in the car so these are does not matter which subtask you are doing this in these are all wrong.

That is part of the core MDP so regardless of which sub sub-task you are doing this in it will be wrong so that is what so picking up at the wrong location dropping at the wrong location all of this are bad and like he was saying if you want to make sure you are getting a smooth ride you can also penalize it for running into obstacles that will be in the core MDP, what could possibly, that is what I am saying if your goal is to have a smooth ride.

Otherwise what will happen if you run into an obstacle you will get a -1 anyway so you will try to avoid running into obstacles so every time step you get a -1 right whether you move or not if you run into an obstacle you do not move so you get a -1 so if you want to really avoid going anywhere near an obstacle then you can give it a high negative reward so that you well basically you cannot reach any of the goals.

Because you have to go very close to an obstacle to reach the goal, so we have to think about those things that is fine okay so what about the transitions and the rewards I have erased the rewards also because that was incorrect what I wrote was not correct, if Ai the Ai that I pick happens to be primitive action then all of it is fine so essentially it is a core MDP transition right and then the core MDP reward function for taking that action.

But if I happen to pick another sub task right a non primitive action as my Ai not Ai Ai is the bigger thing right so the A that I pick at this time instance is a non primitive action so what will be my transition so now I will have to define a probability over right so N is the number of time

steps it is going to take s ' is the final state you are going to land up in s , a right so I will have to do this for task i, what is that n is the number of steps it will take to complete yeah.

If you want I will use tau it is the same it is a dummy variable right so if I used tau earlier I will use tau now right I mean Dietterich uses N so I used N again but you can use tau that is exactly like tau in fact now it is tau t so how will you determine this, this will be determined by how many time steps the subtasks corresponding to a will take to complete okay, so this is determined by Ma where Ma is the a th machine here the a th subtask, correct.

Are people on board okay now what about Ri, we know all about Ri it is the summation of the rewards that you are going to get while the actions corresponding to a are being executed right there will be the Ma will run right so the policy in Ma will now run when I call a, I am going to execute a policy so for every step along the way I am going to get some reward right I am just going to accumulate that.

That is essentially what my Ri will be there will be a discounting factor depending on how we are doing it if you want to use discounted rewards use the discounting factor right so the tricky thing is what should Ri be, think of the MDP definitions what is Ri it is the reward expectation so what I just described to you as a sample you have to convert that into expectation you have to write out the proper transition probabilities and other things.

In order for you to define that right. So it turns out conditioning on s ' and tau is a little tricky right when you write these reward functions so I can write it without conditioning on s ' and tau experimental so that is the expected reward for taking action a in state s right so this is the expectation of this summation given that I started in state s and I did a as my first action and I followed my policy πso now it is little tricky.

So this expected reward that I plug in to solving Mi already has the policy πthere because I need to know what is the policy I am following below Mi for me to compute this reward right because I have called a here but then a might call some other action a ' a ' might call some other action a

double ' right so for example root I might have called pick up but pickup might call navigate and navigate might call south.

So that might be the one action that gets executed maybe it finishes navigate then it will come back to pick up and then it will come back to root right so I basically need to know what was the entire policy that was executed for me to compute this expectation right so I will condition it on the whole πthis is reward for i sorry these are the rewards for the core MDP you are right right now I defined it as the rewards for the core MDP.

In fact I can which one then $\widehat{Ri}$ $\overline{Ri}$ so that is one tricky part because $\overline{Ri}$ will be defined on the states of this MDP right this SMDP while these these small transitions that I see here might be transitions in the smaller SMDP below me right so they might not correspond to the states here I mean they might correspond to different set of states correct so they might correspond to different set of states or it is not entirely clear to me that.

This thing can , I can I can assign a pseudo reward to every step along the way so what I should possibly do is add a separate term for pseudo reward here so that I construct my entire reward function, so this will be the core MDP rewards because these are coming from the subtasks below me right so during the execution of subtasks below me then I could potentially add a pseudo reward right.

Right now I am going to remove the pseudo reward because it adds a lot of complications so most of the development that I will show you now is without the pseudo reward and how do you do the corresponding things along with the pseudo reward I am going to ask you to read the paper essentially you will have to maintain a second value function throughout so one value function which actually uses the only the core MDP values the core MDP rewards.

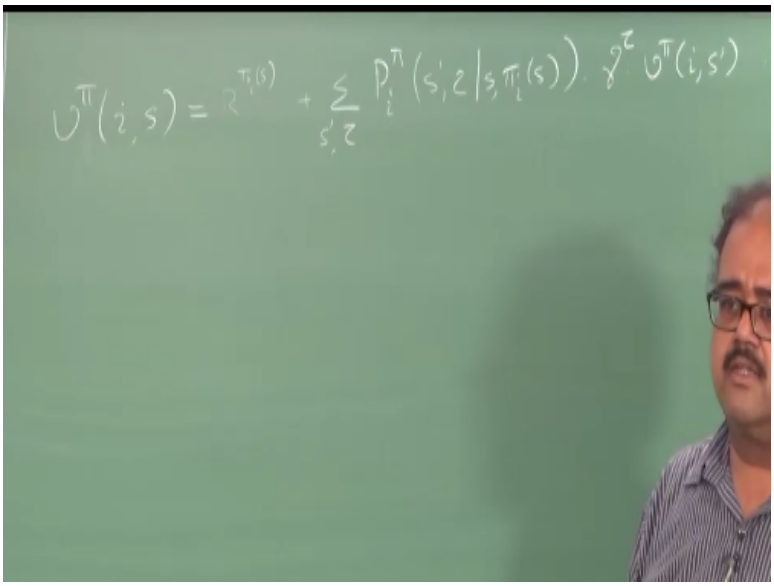
another value function which is used only internally to the sub task which uses the pseudo rewards as well, sorry, no that is the core MDP reward pseudo reward tells you well if you are in pickup right to successfully complete the pickup task you use both the core MDP and the pseudo reward right but finally when you want to solve the problem.

But you do not use the pseudo reward of the navigate while you are trying to solve the pickup task that is the difference when I am trying to solve root so what will be the pseudo reward of root, nothing because the root is trying to solve the original problem so the pseudo the reward for root should be the core reward function right, so the reward for root is the core reward function and it will never use the pickup pseudo reward while solving this right.

Likewise pickup will never use navigate pseudo reward navigate will never use pickup pseudo reward right the pseudo reward is limited only to that sub task for which you have defined the pseudo reward okay so just just ignore the pseudo reward we will just we will come back to that later, right. So what I am going to do now now that we have some kind of definition for the SMDP okay.

Let us start writing down the value functions okay then we can worry about how will we learn those value functions right.

(Refer Slide Time: 15:59)



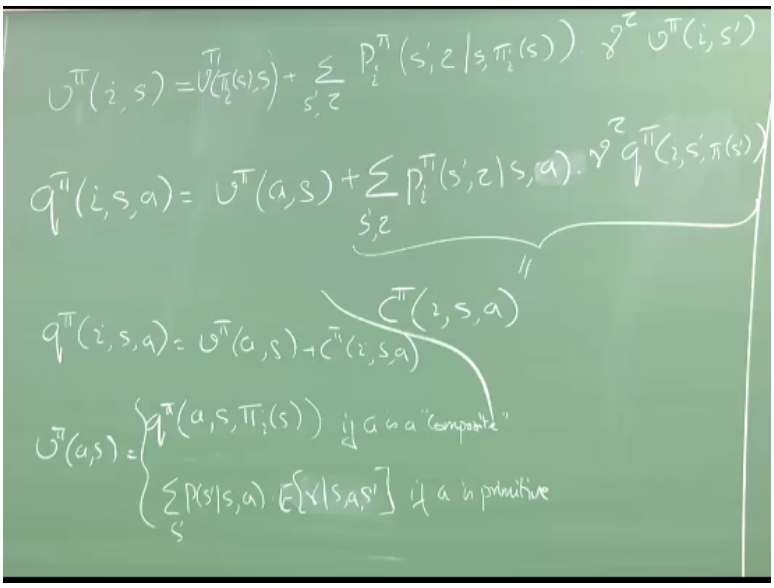
So that is essentially what we are looking at so I am going to use the following notation so I am going to say v πI , s is the value function of state s in task i right I mean I could have done i here right I could have said this is the value function corresponding to task i I mean I would have said that is a more natural notation but I am not putting i there I am putting i here because that is way Dietterich defined it.

So it is easier for you to follow when you look at the paper later, right. So what do you think this will be you want to do bellman style writing up right how will I do this so I will say first say pick the cost corresponding to the first action you are doing right which will be let us say πright so that will be the first action I am doing right and then right so that is like the bellman equation. So what is missing here, the Σ pi.

Why is it missing, I heard something some other murmurs also let us see if someone gave the right answer why is the summation of πmissing, Oh come on guys wake up we did a version of this already because I am assuming πis deterministic that is why I wrote πas, heavy lunch today, noticing too many people with half glazed eyes Σi that is missing yeah fine one of a Σ a is Σ a, πs a, yeah.

Why did not you say Σ a is missing you are trying to figure what πis, is it okay just say Σ a is missing okay that is because you are assuming it is a deterministic policy right why what is next why would it go to a different SMDP because I am just looking at the value of state s and i, no no he is right he is right so this is too much notation in MaxQ.

(Refer Slide Time: 20:00)



So one of the things which I should have introduced earlier so we have value functions corresponding to the actual policy πright so we have value functions corresponding to actual policy πright and so that is going to look at the entire execution of the policy right so I am going to call define something called the projected value function which looks at the value function only for till the completion of task i, right so I am only worried about task i finishing I am not worried about what happens after task i.

Right so this is the projected value function and what we are writing here is the bellman equation for the projected value function okay infact in a later paper somebody pointed out that there is a problems with the Max Q architecture the way the Max Q value function decomposition the way Dietterich defined it precisely because he operates with the projected value functions right because he operates with the projected value functions.

there is no chance of you learning a hierarchically optimal policy so they came up with the different kinds of value function decomposition that allows you to choose to learn the recursive optimal policy or a hierarchical optimal policy but you need to have a different value function decomposition that did not just operate with the projected value function.

But it had something that came after the projected value function also I will explain what in a minute but what we are talking about is a projected value function and you are right that I am only worried about till the task i completes I do not care about what happens after i sorry right given that caveat is it all clear what is happening here right but what is this guy not pseudo reward think about it yeah over which task Noo..the πi th subtask, πi of s, right I would have taken some action there right.

So the entire reward I accumulate over the execution of that action should go in here right and what is that quantity do not look here look at the notation we have introduced already here right, so it is the value function in that sub projected value function in that subtask so what is the projected value function it is the total reward I accumulate till the end of that option the end of that sub task right so now what happens when I call πi of s it will run till that completes and then it will come back.

So what is the projected value function in πi of s, it is essentially the reward I have accumulated till the end of πi of s right so similarly I can write in sub task i what is the value function s , a, right this is even clearer to write, this is essentially right do not get confused now I have flipped a and s this is why I said I do not like this notation if I had written a here it would have been very clear right so here you have s a here it is a s, that essentially means it is the value of state s in subtask a just like this is value of state s in subtask i this is value of state s in subtask a right and then what else do you have essentially have the same thing.

If you want to write that last term in terms of q what will I do this is q πso I have now bellman equation for V πand a bellman equation for q pi, so sometimes not sometimes so what we do this we define I am really off color today so we call this thing the completion function C a little

confusing here so C π(i s a) is the cost or pay off or reward that you are going to accumulate in sub task i after you complete action a starting from state s right.

So look at the way this works right I have completed action a once I complete action a will be in some s ' so I am looking at how much reward I am going to accumulate from s ' till the end of sub task i, so the completion function exactly says that so I started a in state s after I finish that how much more reward will I get till the completion of sub task i, so that is called the completion function so I can write my q function as so my q is essentially v + C right.

So C is equal to this so V πa s equal to q πright, so q πis this v πis okay, okay fine you did not sleep and fall over and so v πis q π of a s πs if it is a composite action basically like one of the subtasks and it is primitive action then it is just the expected value of the reward just because I made this into a s right so this is essentially in what is the value of state s if under primitive action a which is essentially this I am marginalizing over s ' right so this will be what is a value of state s in subtask a right.

Which is essentially the q function so now you can see kind of it goes recursively right so I want to compute the q so I need the C right but for computing the V if it is a composite action I go over go down and ask for the q of the lower action right if it is a primitive action I return this so it is some kind of a hierarchical thing so if I want to so I can make you really really sweat by saying okay, here is the value of state s in 0 , nice I see somebody is awake not me right so how will you write V πo of s.

First let me define some notation for you to write this out so I am going to say πis composed of right so any policy πthat I define is a hierarchical policy and it is composed of π0, π1, π2.. till πk which are essentially the policies I will be following in each one of the k subtasks right I have M0 to Mk so π0 is the policy I will follow in M0, π1 is the policy I will follow in M1 and so on so forth so πis written out like this right now I want you to write out the value function for state s in subtask 0 right.

So how will it look like so what will happen is πnaught will call some other action below it first action it will call is something below it right so let us say it the first action called by πnaught is a one correct so what I will essentially do is I will get I am going to write it slightly ulta (meaning reverse) so that it is easier for me to keep adding terms to the end so what should I have v π0 would be q π0 s , a1 because I said a1 will be the first action I take right.

So πa of s.. πo of s will be a1 let us say right so q π0 , s , a1 right and then what do I do, I add the right so I will essentially for that I will have to for computing that q function what should I do zero , s , a1 so that will be v π of a1 , s + C π(0 , s , a1) + right. Now let us say that I go into a1 okay and let us say for sake of ease of writing the first action I take in a1 is a2, so now what will I do it is a composite action right v πa1 , s will be q πa1 , s ,.

Which is a2 right so q πa , s , a2 right and then I will go back to q πa , s , a2 I mean a1 sorry a1 , s , a2 which will be v πa2 , s + C π + C πa1 , s , a2, all the pi's will be πright these are all conditioned on the whole policy that I am executing right so πnaught of s is a1 π1 of s is a2 π2 of s is a3 that is what we will say yeah within the subtask I will be choosing a πthen what will come here it will essentially be π of a2 , s right so like that I will keep going until I hit a primitive action right.

So if I want to compute v π0 of s so essentially I will make I look at what the action I pick is right I will make a query I will look at the completion function for that action then I will go down look at the completion function of the choice made in the subtask then if I am if I reach the end and then I will just use the v πright if I reach the end sorry if I reach the end I will just use this expression right otherwise I will use the q and I keep going right so this is essentially what the value function decomposition is what is the nice thing about this is.

If I take a2 for some other subtask also right here I am looking at v π0 , s suppose I look at v π0 , s ' right, so there also I would have learnt this C πI mean this is the same C πthat I will be using regardless of how I come to the end right I might start from s and go to some state if I might start from s ' so well I have to learn different this thing but the point is if you go further down the hierarchy it makes more sense not in the root.

So further down the hierarchy there could be multiple ways in which you could have started the subtask right I would have learnt the same completion function I can just reuse the completion function everywhere likewise I can reuse the yeah so if I am caching the q function I can reuse the q function also everywhere right so essentially what this allows me to do is break down the value function right into chunks of rewards right.

Which I am likely to see again and again right each chunk corresponds to executing a subtask so that is essentially what value function decomposition does and yeah so we are already getting into confusion right this is really all is a pretty complex setup already without bringing in the pseudo rewards there are no pseudo rewards here right already the completion function and this becomes a pain so if you look at the root task right everything is good in the root task what did we have so we had the completion task a completion cost or the completion function after you finish a1 starting from s for completing the root task.

What is the reward so that is all fine right but then if you go to a sub level so what it what is the completion function here say only for completing a1 I do not care what happens after a1 right but in reality if I want to learn something hierarchically optimal right I need to know what is going to happen after a1 if you think about it right so it is not enough for me to get out of the door I need to know what happens after I get out of the door so that I can decide which door to get out of right so I need to know what happens after the completion of a1 also.

So what was proposed as a modification was a second completion function which was for the whole problem right but then it becomes tricky how do you maintain this across the entire structure right how do you maintain this consistent essentially it is like passing on information from some sub task very far down the line all the way up to the first sub task right suppose I am executing subtask one and my task ends after subtask hundred then whatever happens till 100 all that information has to be brought back to sub task one for me to define.

The second completion function.Right so how do you do efficient bookkeeping so that is essentially the problem that they had to solve for learning hierarchical optimal policies with the

MaxQ so right now you can define these kinds of completion function and now you can define a q learning SMDP Q learning variation right where at every sub task you update the q value for the subtask as well as the completion value right.

So you do not update the q value of the subtask you just keep calling down till you reach a primitive action so you basically have this right as we saw here right so I can keep calling down until I reach a primitive action so I am not yet reached the primitive action here but if I reach a primitive action then basically all I have is this right and this I do not know because this assumes that I have knowledge of p and the knowledge of this expectation so what should I do for this maintain some kind of a running average or something right.

So essentially that is what you will be doing so if you hit a primitive action you will return that running average so otherwise you keep querying so all you really need to store is that running average which is essentially the q function the q value right if you think about it the running average is essentially the q value for taking action a in state s and what is this the q value for taking action a in state s right this expression I have written here the original q value no SMDP nothing the original core MDP q value for taking action a in state s.

So it is Q s a is that right yes or no, Noo yeah q value have the future expectations right so it is just the expected reward for taking action a in state s right so that is all this is so you basically compute the expected reward for taking action a in state s and that is one thing that you have to remember and apart from that what else you have to remember for every action you will have to remember the completion cost. So what you now you basically define an SMDP q learning variation where you update the completion cost at every state right.

Whenever you take an action and whenever you come to a primitive action you update the expected reward whenever you take a composite action you update the completion function whenever you take a primitive action you update the so essentially that is roughly what it will reduce to so there is something called Max q0 learning right so Max q0 learning is for the cases where the pseudo reward is zero right uniformly zero for all sub tasks in all states the pseudo

reward is zero then you have Max q0 learning which is essentially what I described to you you update the completion function and the expected reward and that is basically it.

Then you have MaxQ q learning where it is defined for arbitrary pseudo reward functions where you maintain two completion functions one for solving the subtask one one completion function which you use for picking actions in the sub task and one completion function which you use for answering queries from above right, so when pickup asks navigate give me your completion function navigate will send the function that was learnt only with the core rewards.

But when navigate wants to know the Q values to pick an action it will use the completion function learnt with the pseudo rewards okay make sense, so you have to but then the book keeping becomes more more painful so you have to make sure you are updating the right q functions right completion functions throughout so that is essentially what MaxQ q learning is all about so yeah we are done with MaxQ today.

IIT Madras Production

Funded by
Department of Higher Education
Ministry of Human Resource Development
Government of India

www.nptel.ac.in